

Guía de Actividades Práctico-Experimentales Nro. 001

1. Datos Generales

Integrantes	Santiago Villamagua Mauricio Guachizaca Mateo Silva María Valarezo Cristopher Chingo
Asignatura	Sistemas Digitales
Ciclo	5to
Unidad	1 – Microcontroladores para sistemas embebidos
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	1
Título de la Práctica	Uso de Wokwi y Microchip Studio en microcontroladores AVR: arquitectura, puertos de E/S y operaciones a nivel de bit
Nombre del Docente	Luis Darío Sinche C.
Fecha	02 de octubre de 2026
Horario	Jueves 12h30 a 13h30 Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (práctica 100 % simulada)
Tiempo planificado en el Sílabo	3 horas (APE, semana 1)
Modalidad de trabajo	En parejas, con simulación en Wokwi y Microchip Studio

2. Objetivo(s) de la Práctica:

Objetivo general. Reconocer la arquitectura del microcontrolador ATmega328P y controlar sus periféricos de entradas/salidas digitales escribiendo y leyendo directamente sus registros mediante operaciones a nivel de bit, usando simulación en Wokwi y Microchip Studio.

Objetivos específicos:

- Relacionar los pines de la placa Arduino UNO con los puertos y bits del ATmega328P (PB, PC, PD).
- Configurar entradas y salidas digitales con los registros DDRx, PORTx y PINx usando máscaras (AND, OR, XOR, NOT y desplazamientos).
- Comparar el acceso mediante la API de Arduino con el acceso directo a registros, en tiempo de ejecución.
- Identificar en el código ensamblador generado las instrucciones

lógicas y de manejo de bits del set de instrucciones AVR (SBI, CBI, SBIS, IN, OUT, ORI, EOR).

3. Fundamento teórico

Un **microcontrolador** integra en un solo chip la CPU, la memoria de programa, la memoria de datos y los periféricos (puertos de E/S, temporizadores, UART, ADC...); por eso es la base de un **sistema embebido**: un computador dedicado a una tarea concreta dentro de un producto. Un microprocesador, en cambio, necesita memoria y periféricos externos.

El **ATmega328P** (placa Arduino UNO) es un AVR de 8 bits con arquitectura **RISC** y **Harvard** (buses separados para programa y datos): 32 KB de memoria Flash, 2 KB de SRAM, 1 KB de EEPROM y 32 registros de propósito general (R0–R31). La mayoría de sus instrucciones se ejecuta en un ciclo de reloj; en el Arduino UNO el reloj es de 16 MHz (62,5 ns por ciclo).

Cada pin digital pertenece a un **puerto** (B, C o D) y se controla con un bit en **tres registros**:

Tabla 1. Registros de un puerto de E/S del ATmega328P

Registro	Función	Bit = 0	Bit = 1
DDR _x	Dirección del pin	Entrada	Salida
PORT _x	Si es salida: nivel. Si es entrada: resistencia pull-up	Nivel bajo / sin pull-up	Nivel alto / pull-up activa
PIN _x	Lectura del nivel real del pin	Pin en 0 V	Pin en VCC

En el ATmega328P, escribir un 1 en un bit de **PIN_x** **conmuta** el bit correspondiente de **PORT_x** (consulte la hoja de datos, sección de puertos de E/S).

Para modificar un bit sin alterar los demás se usan **máscaras**:

Tabla 2. Patrones de manipulación de bits

Operación	Expresión en C	Idea
Poner a 1 el bit n	<code>REG = (1 << n);</code>	OR con una máscara que solo tiene el bit n en 1
Poner a 0 el bit n	<code>REG &= ~(1 << n);</code>	AND con la máscara invertida
Conmutar el bit n	<code>REG ^= (1 << n);</code>	XOR: invierte solo donde la máscara tiene 1
Leer el bit n	<code>(REG >> n) & 1</code>	Desplazar y aislar el bit menos significativo
Escribir un campo	<code>REG = (REG & ~M) (v & M);</code>	Limpiar el campo M y luego cargar el valor

Correspondencia en el Arduino UNO: **D0–D7** → **PD0–PD7** (D0 y D1 son RX/TX de la UART), **D8–D13** → **PB0–PB5** (D13 tiene el LED integrado "L") y **A0–A5** → **PC0–PC5**.

4. Materiales y reactivos:

No se requieren componentes físicos: toda la práctica es simulada. Componentes virtuales usados en Wokwi:

- 1 Arduino UNO (ATmega328P a 16 MHz).
- 4 LEDs (rojo, amarillo, verde y azul) y 4 resistencias de 220 Ω.
- 1 pulsador.

5. Equipos y herramientas

- Computador con navegador web y conexión a internet.
- Wokwi (<https://wokwi.com>), con cuenta gratuita para guardar y compartir los proyectos.

- Microchip Studio 7 (solo Windows), con el simulador del ATmega328P. Si no dispone de Windows, use Compiler Explorer (<https://godbolt.org>) para la Parte E.
- Hoja de datos del ATmega328P y AVR Instruction Set Manual (Microchip Technology).
- Cuenta de GitHub y acceso al EVA para la entrega.

6. Procedimiento / Metodología

Trabajo en **equipos**. Regla general: desde la Parte B **no se usan** `pinMode()`, `digitalWrite()` ni `digitalRead()`; todo el control de pines se hace por registros. Duración total: 3 h.

Parte A. Reconocimiento del entorno y del microcontrolador

- A1.** Ingrese a wokwi.com, cree un proyecto nuevo con **Arduino Uno** y guárdelo con el nombre `APE1_<apellido1>_<apellido2>`.
- A2.** En la placa simulada, identifique el chip ATmega328P, el LED integrado "L" y el pin D13. Con la sección de Fundamento teórico, indique a qué puerto y bit corresponde D13.
- A3.** Escriba el Código 1 (línea base con la API de Arduino), inicie la simulación y verifique que el LED "L" parpadea.

Código 1. Blink con la API de Arduino

```
// APE 1 - Código 1: Blink con la API de Arduino (línea base)
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
void setup() {
  pinMode(13, OUTPUT);          // D13 = PB5 (LED integrado "L")
}

void loop() {
  digitalWrite(13, HIGH);
  delay(500);
  digitalWrite(13, LOW);
  delay(500);
}
```

Parte B. El mismo comportamiento, ahora con registros

- B1.** Reemplace el código por el Código 2 y verifique que el comportamiento es idéntico al del Código 1.

Código 2. Blink escribiendo registros

```
// APE 1 - Código 2: el mismo Blink, ahora escribiendo registros
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
// Regla: no se usa pinMode() ni digitalWrite()
void setup() {
  DDRB |= (1 << DDB5);          // PB5 (D13) como salida; los demás bits no cambian
}

void loop() {
  PORTB ^= (1 << PORTB5);        // XOR: conmuta solo el bit 5
  delay(500);                    // 500 ms encendido / 500 ms apagado
}
```

- B2.** Sustituya `PORTB ^= (1 << PORTB5);` por un par de líneas que enciendan (`|=`) y apaguen (`&= ~`) el LED con un `delay(500)` entre ellas. Verifique.
- B3.** Sustituya la conmutación por `PINB = (1 << PINB5);` y verifique. Anote qué característica del ATmega328P lo permite.
- B4.** Pruebe `PORTB = (1 << PORTB5);` (asignación en lugar de OR). Funciona aquí, pero explique en el informe qué les pasaría a los otros bits de PORTB si hubiera más salidas conectadas en ese puerto. Lo comprobará en la Parte C.

Parte C. Circuito y laboratorio de bits

C1. Arme en Wokwi el circuito de la Tabla 3. Puede hacerlo a mano o pegar el contenido del Anexo A en la pestaña `diagram.json`.

Tabla 3. Conexiones pin a pin

Componente	Pin del componente	Pin Arduino UNO	Puerto / bit	Configuración
LED0 (rojo) + R0 220 Ω	Ánodo (A) vía R0	D8	PB0	Salida
LED1 (amarillo) + R1 220 Ω	Ánodo (A) vía R1	D9	PB1	Salida
LED2 (verde) + R2 220 Ω	Ánodo (A) vía R2	D10	PB2	Salida
LED3 (azul) + R3 220 Ω	Ánodo (A) vía R3	D11	PB3	Salida
LED0 ... LED3	Cátodo (C)	GND	—	—
Pulsador	1.I	D2	PD2	Entrada con pull-up interna
Pulsador	2.I	GND	—	—
LED integrado "L"	—	D13	PB5	Salida (latido)

Nota sobre pines: no use D0 ni D1, porque son la UART que usa el Monitor Serie. El pulsador va a GND porque se usa la resistencia pull-up interna, así que se lee 1 suelto y 0 presionado (activo en bajo). En el `diagram.json` el rebote del pulsador está desactivado (`"bounce": "0"`); en hardware real habrá que tratar el rebote.

C2. Cargue el Código 3, inicie la simulación y abra el Monitor Serie (9600 baudios).

Código 3. Laboratorio de bits (Arduino UNO en Wokwi, sin bibliotecas externas)

```

/*
  APE 1 - Código 3: Laboratorio de bits en los puertos B y D
  Placa : Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
  Libs : ninguna (avr/io.h llega incluido por Arduino.h)
  Pines : PB0..PB3 (D8..D11) LEDs | PD2 (D2) pulsador a GND | PB5 (D13) LED L
  Regla : NO se usa pinMode(), digitalWrite() ni digitalRead()
*/
const uint8_t MASK_LEDS = 0x0F; // 0b00001111 -> PB0..PB3
const uint8_t NUM_MODOS = 3;
const uint8_t PATRON_INICIAL[NUM_MODOS] = {0b0000, 0b0001, 0b0101};
const unsigned long PERIODO MS = 300;

uint8_t modo = 0;
uint8_t patron = PATRON_INICIAL[0];
uint8_t btnAnterior = 1; // con pull-up, en reposo se lee 1
unsigned long tAnterior = 0;

void imprimirBin8(const char *etiqueta, uint8_t v) {
  Serial.print(etiqueta);
  for (int8_t i = 7; i >= 0; i--) {
    Serial.print((v >> i) & 1); // extrae el bit i
  }
  Serial.println();
}

void escribirLeds(uint8_t valor) {
  // leer-modificar-escribir: solo cambian PB0..PB3; PB4..PB7 se conservan
  PORTB = (PORTB & ~MASK_LEDS) | (valor & MASK_LEDS);
}

void demoOperaciones() {
  uint8_t a = 0b11001100;
  uint8_t b = 0b10101010;
  imprimirBin8("a", a);
  imprimirBin8("b", b);
  imprimirBin8("a & b", a & b);
  imprimirBin8("a | b", a | b);
  imprimirBin8("a ^ b", a ^ b);
  imprimirBin8("~a", (uint8_t)~a);
  imprimirBin8("a << 1", (uint8_t)(a << 1));
}

```

```

imprimirBin8("a >> 2 = ", a >> 2);
Serial.print("~a sin cast, en BIN: ");
Serial.println(~a, BIN);                                     // ver pregunta de control 6
}

void setup() {
  DDRB |= MASK_LEDS | (1 << DDB5);                          // salidas: PB0..PB3 y PB5
  DDRD &= ~(1 << DDD2);                                       // PD2 entrada (bit DDR = 0)
  PORTD |= (1 << PORTD2);                                     // pull-up interna (bit PORT = 1)

  Serial.begin(9600);
  Serial.println(F("APE 1 - Laboratorio de bits"));
  demoOperaciones();
  escribirLeds(patron);
}

void loop() {
  // 1) Pulsador: se aísla el bit 2 de PIND (activo en bajo)
  uint8_t btn = (PIND >> PIND2) & 1;                         // 1 = suelto, 0 = presionado
  if (btnAnterior == 1 && btn == 0) {                         // flanco de bajada
    modo = (modo + 1) % NUM_MODOS;
    patron = PATRON_INICIAL[modo];
    Serial.print(F("Modo: "));
    Serial.println(modo);
  }
  btnAnterior = btn;

  // 2) Actualización periódica sin bloquear el programa
  if (millis() - tAnterior >= PERIODO_MS) {
    tAnterior = millis();
    switch (modo) {
      case 0:                                                  // contador binario de 4 bits
        patron = (patron + 1) & MASK_LEDS;
        break;
      case 1:                                                  // rotación circular a la izquierda
        patron = ((patron << 1) | (patron >> 3)) & MASK_LEDS;
        break;
      case 2:                                                  // alternancia con XOR
        patron ^= MASK_LEDS;
        break;
    }
    escribirLeds(patron);
    PINB = (1 << PINB5);                                       // escribir 1 en PINB5 conmuta PB5
    imprimirBin8("PORTB = ", PORTB);
  }
}

```

- C3.** Antes de mirar el Monitor Serie, calcule a mano los resultados de `demoOperaciones()` y luego compárelos con la salida.
- C4.** Presione el pulsador y describa los tres modos. Para cada modo, explique qué operación a nivel de bit produce el patrón.
- C5.** Observe la línea `PORTB = ...` del Monitor Serie e identifique qué bits corresponden a los LEDs y cuál al LED "L".
- C6.** Cambie `escribirLeds()` por `PORTB = valor;` y describa qué ocurre con el LED "L". Relacione el resultado con el paso B4 y restaure la versión original.
- C7.** Complete la Tabla 4 en el informe.

Tabla 4. Configuración de registros (para completar)

Pin	Función	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido			—
D11 (PB3)	LED3 apagado			—
D2 (PD2)	Pulsador suelto			
D2 (PD2)	Pulsador presionado			
D13 (PB5)	LED "L" encendido			—

Parte D. Costo de la API frente al acceso a registros

- D1.** En un proyecto nuevo de Wokwi (Arduino Uno, sin componentes adicionales), cargue el Código 4 y ejecútelo tres veces.

Código 4. Medición de tiempo de escritura

```
// APE 1 - Código 4: costo de digitalWrite() frente al acceso directo al registro
// Placa: Arduino UNO (ATmega328P @ 16 MHz) - simulacion en Wokwi
const uint16_t N = 10000;

void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);

  unsigned long t0 = micros();
  for (uint16_t i = 0; i < N; i++) {
    digitalWrite(13, HIGH);
    digitalWrite(13, LOW);
  }
  unsigned long t1 = micros();
  for (uint16_t i = 0; i < N; i++) {
    PORTB |= (1 << PORTB5);
    PORTB &= ~(1 << PORTB5);
  }
  unsigned long t2 = micros();

  Serial.print(F("digitalWrite : "));
  Serial.print((t1 - t0) / (2.0 * N), 3);
  Serial.println(F(" us por escritura"));
  Serial.print(F("Registro      : "));
  Serial.print((t2 - t1) / (2.0 * N), 3);
  Serial.println(F(" us por escritura"));
}

void loop() {}
```

- D2.** Registre los tiempos de las tres ejecuciones, calcule el promedio y la relación digitalWrite/registro. Los tiempos incluyen la sobrecarga del bucle y de la interrupción del Timer0, por lo que interesa la comparación, no el valor absoluto.

Parte E. Del C al set de instrucciones con Microchip Studio

- E1.** En Microchip Studio: **File > New > Project > GCC C Executable Project**, dispositivo **ATmega328P**.
- E2.** Reemplace el contenido de `main.c` por el Código 5 y compile (**Build > Build Solution**, F7). Verifique que no haya errores.

Código 5. C puro para Microchip Studio (sin Arduino)

```
/*
 * APE 1 - Código 5: Microchip Studio (C puro, sin Arduino)
 * Dispositivo: ATmega328P | Toolchain: AVR/GNU C Compiler (avr-gcc)
 */
#include <avr/io.h>

int main(void)
{
  DDRB |= (1 << DDB5) | (1 << DDB0); // PB5 y PB0 como salidas
  DDRD &= ~(1 << DDD2); // PD2 como entrada
  PORTD |= (1 << PORTD2); // pull-up interna en PD2

  while (1)
  {
    if (PIND & (1 << PIND2)) { // pulsador suelto (lee 1)
      PORTB &= ~(1 << PORTB5); // apaga PB5
    } else { // pulsador presionado (lee 0)
      PORTB |= (1 << PORTB5); // enciende PB5
    }
    PORTB ^= (1 << PORTB0); // conmuta PB0 en cada vuelta
  }
}
```

}

E3. Abra el archivo `.lss` (Solution Explorer > Output Files). Localice la función `main` y complete la Tabla 5 con las instrucciones generadas para cada línea de C y sus ciclos según el AVR Instruction Set Manual.

Tabla 5. Correspondencia C – ensamblador (para completar)

Línea en C	Instrucción(es) AVR	Ciclos
<code>DDRB = (1 << DDB5) (1 << DDB0);</code>		
<code>DDRD &= ~(1 << DDD2);</code>		
<code>PORTD = (1 << PORTD2);</code>		
<code>if (PIND & (1 << PIND2))</code>		
<code>PORTB &= ~(1 << PORTB5);</code>		
<code>PORTB = (1 << PORTB5);</code>		
<code>PORTB ^= (1 << PORTB0);</code>		

E4. Seleccione el simulador (**Project > Properties > Tool > Simulator**) e inicie la depuración con **Debug > Start Debugging and Break** (Alt+F5).

E5. Abra **Debug > Windows > I/O**, seleccione PORTB y PORTD y avance instrucción por instrucción (F10/F11). Observe cómo cambian DDRB, PORTB y PORTD. Simule el pulsador modificando el bit 2 de PIND en la vista I/O y observe el efecto en PB5. Si el simulador no refleja la pull-up, ponga PIND2 en 1 a mano para representar "suelto".

E6. En **Debug > Windows > Processor Status**, use el contador de ciclos para medir cuántos ciclos tarda una vuelta del `while(1)` con el pulsador suelto y con el pulsador presionado.

Alternativa sin Windows: en <https://godbolt.org> seleccione el lenguaje C y un compilador "AVR gcc" con las opciones `-O1 -mmcu=atmega328p`, pegue el Código 5 y complete la Tabla 5 con el ensamblador mostrado. Los pasos E4 a E6 se reemplazan por el análisis manual de ciclos. Las instrucciones pueden variar ligeramente según la versión del compilador y el nivel de optimización.

Parte F. Cierre y entrega

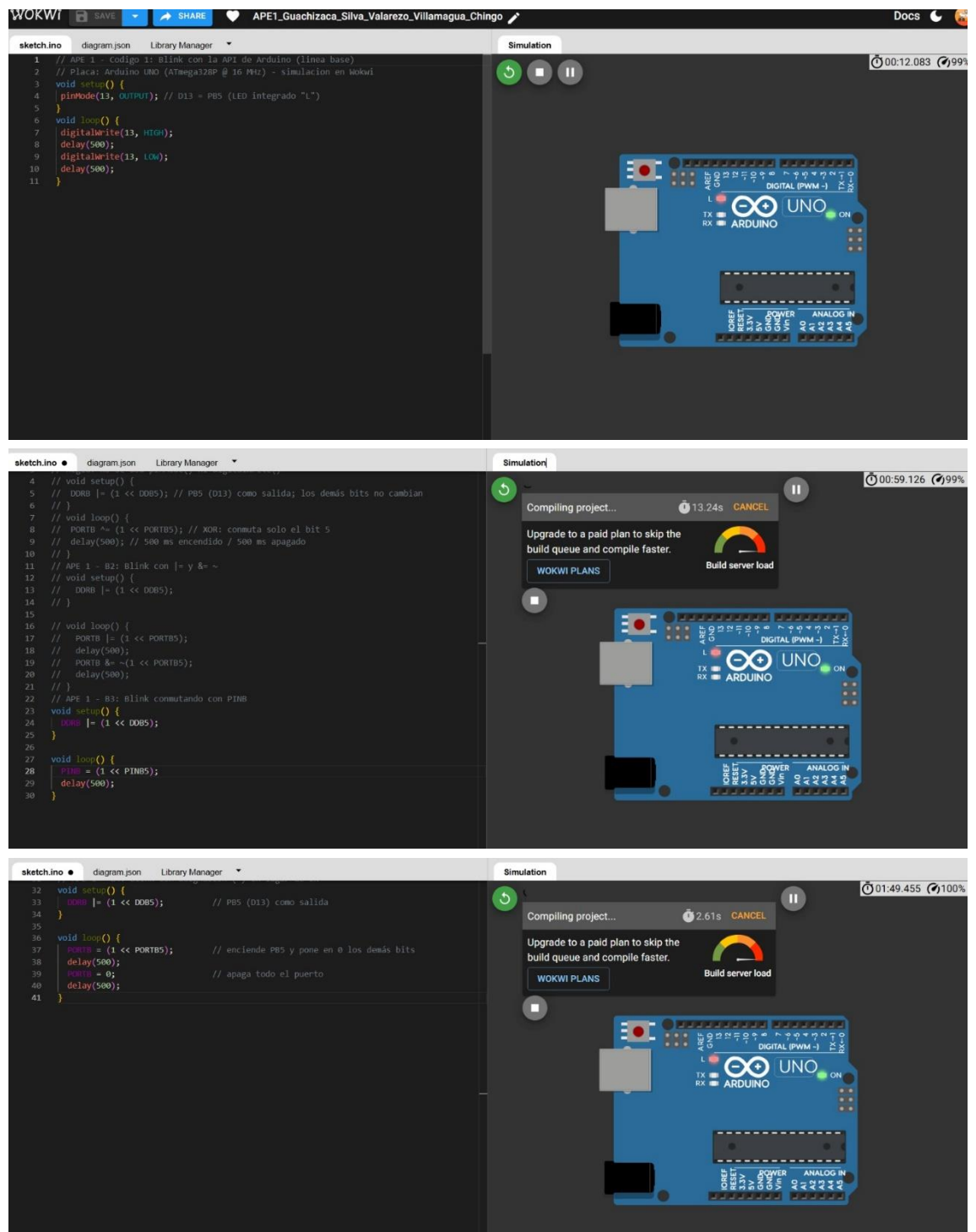
F1. Publique los proyectos de Wokwi como públicos y copie sus enlaces.

F2. Suba los códigos, el `diagram.json` y el informe al repositorio de GitHub de la pareja y entregue el informe en el EVA.

7. Resultados esperados:

- Códigos 1 y 2:** el LED "L" permanece 500 ms encendido y 500 ms apagado (periodo de 1 s), con comportamiento idéntico por API y por registros.

Parte A:

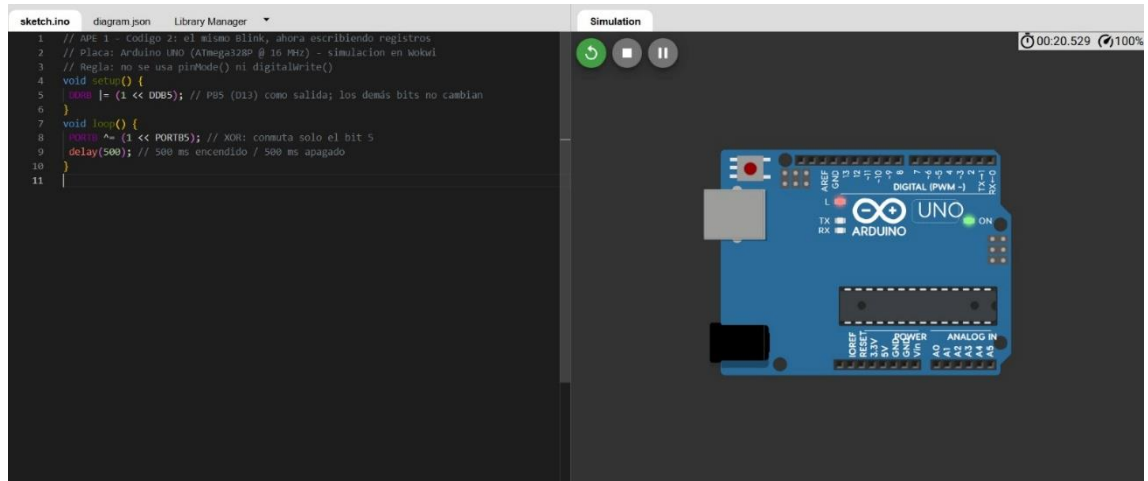


Parte B:

B1.

- Código 2

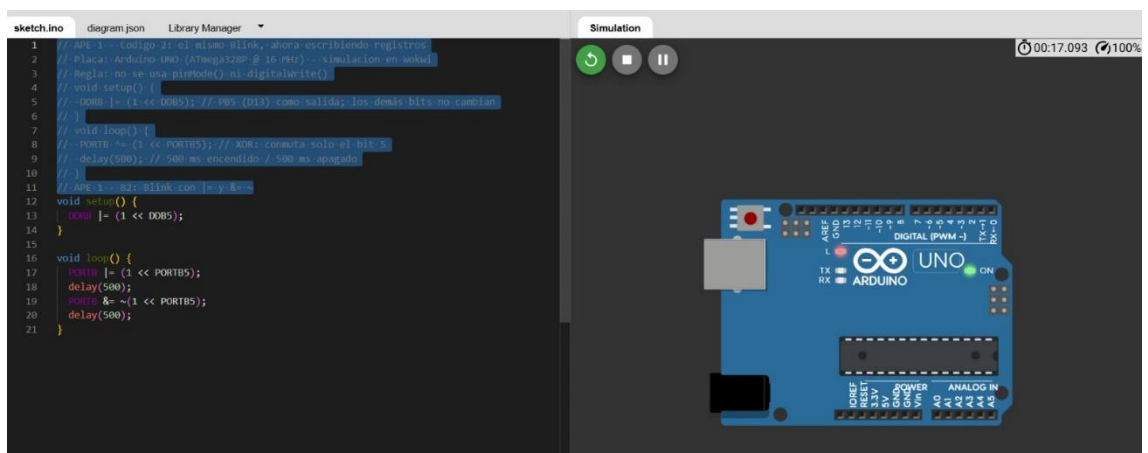
Se reemplazó la API de Arduino por manipulación directa de registros. Se configuró el bit 5 del registro de dirección DDRB en nivel alto (1) para establecer el pin D13 (PB5) como salida. En el bucle principal se utilizó el operador XOR (^=) para alternar el bit 5 de PORTB cada 500 ms.



Se verificó en la simulación de Wokwi que el comportamiento es idéntico al del Código 1 (el LED "L" parpadea a un intervalo de 1 segundo en total).

B2.

Se sustituyó la conmutación XOR por secuencias explícitas de encendido y apagado utilizando máscaras OR y AND invertida:



Se comprobó que el LED parpadea correctamente. Esta estructura permite controlar de forma independiente cuándo se activa y cuándo se desactiva una salida.

B3.

Se reemplazó la conmutación por la línea:

PINB = (1 << PINB5);

El microcontrolador ATmega328P incluye una función a nivel de hardware denominada *Toggling the Pin* (conmutación de pin). Esta característica establece que al escribir un 1 lógico en cualquier bit del registro de entrada PINx, el estado del bit equivalente en el registro de salida PORTx se invierte automáticamente (pasa de 0 a 1 o de 1 a 0), mientras que escribir un 0 no produce cambios. Conmuta el pin PB5 (D13) de forma inmediata. A diferencia de la operación XOR ($PORTB \wedge= (1 << PORTB5);$), esta técnica presenta dos ventajas clave:

- **Mayor velocidad:** No requiere realizar la secuencia típica de "lectura, modificación y escritura", sino que modifica el bit en un solo ciclo de reloj.
- **Operación atómica:** Al ejecutarse en una sola instrucción, se

garantiza que no sea interrumpida por una interrupción del sistema que pudiera alterar los datos del puerto a mitad de proceso.

B4.

Aunque la instrucción:

PORTB = (1 << PORTB5);

cumple la función de encender el LED en D13, no es una buena práctica de programación porque realiza una asignación directa sobre los 8 bits del puerto a la vez. Esto provoca que únicamente el bit 5 pase a 1, mientras que los demás bits (PB0 a PB4, PB6 y PB7) se fuerzan a 0.

Esto acarrea dos problemas graves en circuitos más complejos:

1. Apagado no deseado de salidas: Si existieran otros periféricos o LEDs conectados en el Puerto B (como los LEDs de D8 a D11 en la Parte C), esta línea los apagaría de inmediato en cada ejecución, interfiriendo con el resto del programa.
2. Desactivación de resistencias *pull-up*: En los pines configurados como entradas, un 1 en PORTx mantiene activa la resistencia *pull-up* interna. Al sobrescribir todo el registro con ceros, estas resistencias se desactivan, alterando las lecturas del puerto.

Parte C.

- **Código 3:** el Monitor Serie muestra los resultados de AND, OR, XOR, NOT y desplazamientos en 8 bits. Los LEDs muestran un contador binario de 4 bits (modo 0), una luz que rota (modo 1) y una alternancia 0101/1010 (modo 2). El LED "L" cambia de estado cada 300 ms y cada pulsación cambia de modo.

C3:

1. AND Lógico (a & b)

```
a = 1 1 0 0 1 1 0 0
b = 1 0 1 0 1 0 1 0
-----
& = 1 0 0 0 1 0 0 0 -> (0b10001000)
```

2. OR Lógico (a | b)

```
a = 1 1 0 0 1 1 0 0
b = 1 0 1 0 1 0 1 0
-----
| = 1 1 1 0 1 1 1 0 -> (0b11101110)
```

3. XOR Lógico (a ^ b)

```
a = 1 1 0 0 1 1 0 0
b = 1 0 1 0 1 0 1 0
-----
^ = 0 1 1 0 0 1 1 0 -> (0b01100110)}
```

4. NOT / Inversión de Bits (~a)

```
a = 1 1 0 0 1 1 0 0
-----
~a = 0 0 1 1 0 0 1 1 -> (0b00110011)
```

5. Desplazamiento a la Izquierda (a << 1)

a inicial: 1 1 0 0 1 1 0 0
Desplazando: 1 1 0 0 1 1 0 0 _
Agregando 0: 1 0 0 1 1 0 0 0

(El bit '1' de la izquierda se descarta en 8 bits)

6. Desplazamiento a la Derecha (a >> 2)

a inicial: 1 1 0 0 1 1 0 0
Desplazando: _ _ 1 1 0 0 1 1 (00 descartados)
Rellenando: 0 0 1 1 0 0 1 1

C4: Presione el pulsador y describa los tres modos. Para cada modo, explique qué operación a nivel de bit produce el patrón

Modo 0: Suma 1 al patrón actual y aplica una máscara con AND (0b00001111) para asegurarse de que el contador solo afecte a los primeros 4 bits (LEDs 0 a 3).

Modo 1: Realiza un desplazamiento a la izquierda con << 1. El bit que "se sale" por la izquierda se recupera rotándolo hacia el bit más a la derecha con >> 3 y combinándolos con la operación OR (|).

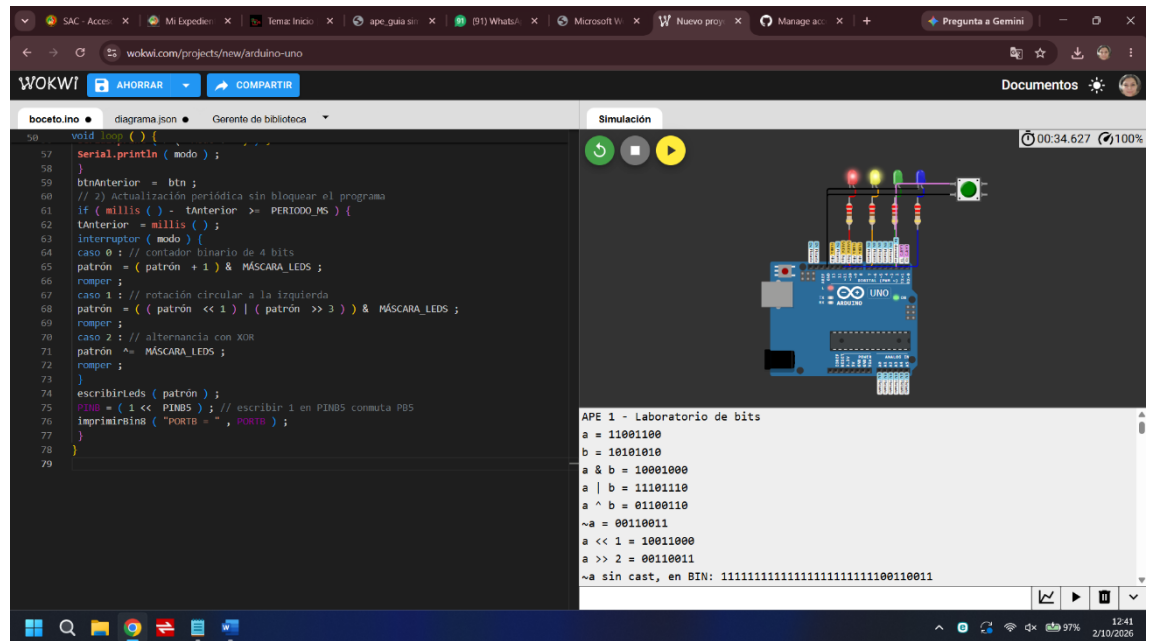
Modo 2: La operación XOR (^) con la máscara 0b00001111 invierte el estado lógico de los 4 bits. Si un bit era 1, pasa a ser 0; y si era 0, pasa a ser 1.

C5: Observe la línea PORTB = ... del Monitor Serie e identifique qué bits corresponden a los LEDs y cuál al LED "L".

- Los 4 bits menos significativos (PB0 a PB3) corresponden a los LEDs 0 a 3.
- El bit PB5 corresponde al latido del LED integrado "L" en D13.

C7:

Pin	Función	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido	1 (Salida)	1 (Alto)	—
D11 (PB3)	LED3 apagado	1 (Salida)	0 (Bajo)	—
D2 (PD2)	Pulsador suelto	0 (Entrada)	1 (Pull-up activa)	1 (Alto por pull-up)
D2 (PD2)	Pulsador presionado	0 (Entrada)	1 (Pull-up activa)	0 (Conectado a GND)
D13 (PB5)	LED "L" encendido	1 (Salida)	1 (Alto)	—



Parte D.

- **Código 4:** el acceso por registro resulta aproximadamente un orden de magnitud más rápido que `digitalWrite()`.

D2. Registre los tiempos de las tres ejecuciones, calcule el promedio y la relación `digitalWrite`/registro. Los tiempos incluyen la sobrecarga del bucle y de la interrupción del Timer0, por lo que interesa la comparación, no el valor absoluto

Ejecución	<code>digitalWrite()</code> (μ s por escritura)	Registro Directo (μ s por escritura)
Prueba 1	3.395	0.220
Prueba 2	3.395	0.220
Prueba 3	3.395	0.220
Promedio	3.395	0.220

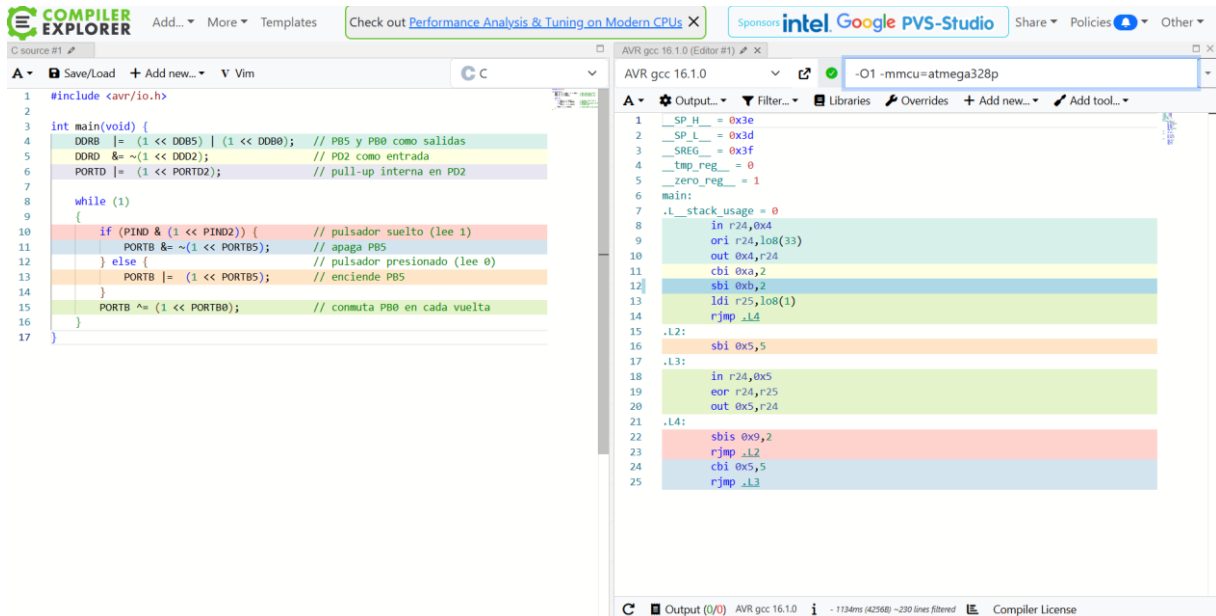
Relación de velocidad:

$$\text{Relacion} = \frac{\text{Promedio Digital Write ()}}{\text{Promedio Registro}} = \frac{3.395 \text{ us}}{0.220 \text{ us}} \approx 15.43$$

El acceso directo a registros resulta 15.43 veces más rápido que la función de la API de Arduino `digitalWrite()`. Esta diferencia se debe a la sobrecarga (overhead) que introduce `digitalWrite()`, la cual en tiempo de ejecución debe mapear el número de pin digital a su correspondiente puerto interno, consultar tablas en memoria Flash y verificar la desactivación de canales PWM antes de escribir. En cambio, el acceso por registros se traduce de forma inmediata en instrucciones atómicas del procesador AVR (SBI / CBI), ejecutándose en apenas 1 o 2 ciclos de reloj.

Parte E.

- **Código 5:** en el .lss aparecen instrucciones de manejo de bits de E/S (SBI, CBI, SBIS) y secuencias de lectura-modificación-escritura (IN, ORI/EOR, OUT).



The screenshot shows the AVR Studio IDE. On the left, the C source code is displayed, featuring a main function that configures DDRB, DDRD, and PORTD, and enters a while loop that checks PIND and manipulates PORTB and PORTD. On the right, the assembly output is shown, corresponding to the C code. The assembly includes instructions like 'in r24, 0x04', 'ori r24, 33', 'out 0x04, r24', 'cbi 0x0a, 2', 'sbi 0x0b, 2', 'sbis 0x09, 2', 'cbi 0x05, 5', 'sbi 0x05, 5', 'in r24, 0x05', 'eor r24, r25', and 'out 0x05, r24'.

Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	in r24, 0x04 ori r24, 33 out 0x04, r24	3
DDRD &= ~(1 << DDD2);	cbi 0x0a, 2	2
PORTD = (1 << PORTD2);	sbi 0x0b, 2	2
if (PIND & (1 << PIND2))	sbis 0x09, 2	1 o 2 si salta
PORTB &= ~(1 << PORTB5);	cbi 0x05, 5	2
PORTB = (1 << PORTB5);	sbi 0x05, 5	2
PORTB ^= (1 << PORTB0);	in r24, 0x05 eor r24, r25 out 0x05, r24	3

Análisis de Tiempo de Ejecución (Contador de Ciclos en Bucle while(1))

Caso 1: Pulsador Suelto (PIND2 = 1 $\rightarrow$ Nivel Alto por Pull-up)

Paso	Operación Realizada	Instrucción AVR	Ciclos de Reloj
1	Evaluar estado del pin PD2	sbis 0x9, 2 (no salta, lee 1)	1
2	Saltar a la rama de apagado del LED	rjmp .L2	2
3	Apagar LED integrando en PB5	cbi 0x5, 5	2
4	Conmutar estado del LED PB0	in, eor, out	3
5	Retorno al inicio del bucle while(1)	rjmp .L4	2
TOTAL	Duración de la iteración	10 u 11 ciclos de reloj	≈ 687.5 ns @ 16 MHz

Caso 2: Pulsador Presionado (PIND2 = 0 $\rightarrow$ Conectado a GND)

Paso	Operación Realizada	Instrucción AVR	Ciclos de Reloj
1	Evaluar estado del pin PD2	sbis 0x9, 2 (salta la siguiente instrucción, lee 0)	2
2	Encender LED integrando en PB5	sbi 0x5, 5	2
3	Conmutar estado del LED PB0	in, eor, out	3
4	Retorno al inicio del bucle while(1)	rjmp .L4	2
TOTAL	Duración de la iteración	9 a 10 ciclos de reloj	≈ 625 ns @ 16 MHz

Se debe entregar el diagrama de conexión, el código realizado, la evidencia de funcionamiento y las tablas completas.

8. Preguntas de Control:

- ¿Qué diferencia a un microcontrolador de un microprocesador? Enumere los bloques del ATmega328P que identificó en la práctica.

Un microprocesador contiene principalmente la CPU y necesita componentes externos, como memoria RAM, almacenamiento y periféricos. Un microcontrolador integra en un solo circuito la CPU, memoria y periféricos para controlar sistemas embebidos.

En el ATmega328P se identifican componentes tales y como CPU de 8 bits AVR, Registros de propósito general, ALU (Unidad Aritmético-Lógica), Memoria Flash para el programa., SRAM para datos, Memoria EEPROM, Puertos digitales de entrada/salida: PORTB, PORTC y PORTD, Registros DDR, PORT y PIN asociados a los puertos, Temporizadores/contadores, ADC (convertidor analógico-digital), Interfaces de comunicación, como USART, SPI e I²C.

Sistema de interrupciones.

2. ¿Qué implica que el ATmega328P tenga arquitectura Harvard? Relacione su respuesta con el uso de la macro `F("...")` en el Código 3.

El ATmega328P utiliza una arquitectura Harvard, lo que significa que la memoria de instrucciones y la memoria de datos están separadas. Esto permite que el microcontrolador maneje el programa y los datos mediante espacios de memoria independientes.

3. Explique por qué `PORTB = (1 << PORTB5);` puede alterar otras salidas y `PORTB |= (1 << PORTB5);` no, usando lo observado en los pasos B4 y C6.

La diferencia está en que `=` reemplaza todo el registro, mientras que `|=` modifica únicamente los bits indicados, en el ejemplo `PORTB = (1 << PORTB5);` genera un valor en el que únicamente queda establecido el bit 5. Los demás bits de `PORTB` se ponen en 0. Por eso puede alterar otras salidas que ya estuvieran activadas.

En cambio en `PORTB |= (1 << PORTB5)` hace una operación OR entre el valor actual de `PORTB` y la máscara:

`PORTB` 10100001

máscara 00100000

resultado 10100001

de modo que los bits que ya estaban en 1 se conservan y únicamente se fuerza a 1 el bit

4. Para usar PD2 como entrada con pull-up, ¿qué valores deben tener `DDRD2` y `PORTD2`? ¿Qué se lee en `PIND2` con el pulsador suelto y presionado, y por qué?

Para utilizar PD2 como entrada con resistencia pull-up interna, el bit `DDRD2` debe estar en 0, ya que esto configura el pin como entrada, mientras que `PORTD2` debe estar en 1 para activar la resistencia pull-up. Cuando el pulsador está suelto, `PIND2` se lee como 1 porque la resistencia pull-up mantiene el pin conectado internamente a un nivel alto. Cuando el pulsador se presiona, este conecta el pin con GND, por lo que `PIND2` pasa a leerse como 0. Por esta razón, el pulsador utiliza una lógica activa en bajo.

5. Calcule a mano `0b11001100 & 0b10101010`, `|` y `^`, y verifique sus resultados con el Monitor Serie.

Al realizar las operaciones bit a bit entre `0b11001100` y `0b10101010`, para AND se obtiene `0b10001000`, ya que solo permanecen en 1 los bits que son 1 en ambos valores. Para OR se obtiene `0b11011110`, porque se establece en 1 cualquier posición donde al menos uno de los valores tenga un 1. Finalmente, para XOR se obtiene `0b01100110`, porque el resultado es 1 únicamente cuando los bits de cada posición son diferentes. Por lo tanto, los resultados son: `& = 0b10001000` (136), `| = 0b11011110` (238) y `^ = 0b01100110` (102). Estos valores pueden verificarse mediante el Monitor Serie de Wokwi.

6. Si `a` es de tipo `uint8_t`, ¿por qué `Serial.println(~a, BIN)` imprime muchos más de 8 bits? ¿Cómo lo corrige el Código 3?

Aunque `a` sea de tipo `uint8_t`, el operador `~` está sujeto a las promociones enteras de C/C++, por lo que el valor puede convertirse temporalmente a un tipo entero de mayor tamaño antes de realizar el complemento. Por eso, al ejecutar `Serial.println(~a, BIN)`, pueden aparecer muchos más de 8 bits. El Código 3 corrige este comportamiento convirtiendo nuevamente el resultado a `uint8_t`, por ejemplo mediante `(uint8_t)~a`, haciendo que solo se conserven los 8 bits correspondientes al tipo original.

7. Con los datos de la Parte D, explique por qué `digitalWrite()` es más lento que el acceso directo a registros. ¿Cuándo conviene cada enfoque (portabilidad frente a rendimiento)?

`digitalWrite()` es más lento que el acceso directo a registros porque es una función de alto nivel que necesita realizar operaciones adicionales para determinar qué puerto y qué bit corresponden al número de pin utilizado. En cambio, una instrucción como `PORTB |= (1 << PORTB5);` trabaja directamente sobre el registro del microcontrolador, por lo que requiere menos operaciones y puede ejecutarse con mayor rapidez. `digitalWrite()` resulta conveniente cuando se busca mayor facilidad de programación y portabilidad, mientras que el acceso directo a registros es más apropiado cuando se necesita mayor rendimiento, tiempos precisos o un control más directo del hardware.

8. ¿Qué instrucción AVR generó `PORTB |= (1 << PORTB5);` y cuántos ciclos toma? ¿Por qué `PORTB ^= (1 << PORTB0);` no se tradujo a una sola instrucción?

Generó la instrucción `sbi 0x05, 5` (Set Bit in I/O Register). Esta instrucción escribe directamente un 1 lógico en el bit 5 del registro de puerto de $\$/S\$$ `PORTB` (dirección `0x05`) y ejecuta en 2 ciclos de reloj.

¿Por qué `PORTB ^= (1 << PORTB0);` no se tradujo a una sola instrucción?

Porque el conjunto de instrucciones (ISA) de los microcontroladores AVR no posee una instrucción atómica directa para aplicar una operación XOR (^) sobre un registro de $\$/S\$$. Por este motivo, el compilador debe generar una secuencia de 3 instrucciones de tipo "lectura-modificación-escritura": `in` (para leer el estado actual del puerto a un registro de trabajo `r24`), `eor` (para aplicar la operación XOR con la máscara en la ALU) y `out` (para escribir el resultado de vuelta en el puerto).

9. Sostenibilidad: indique dos formas en que lo trabajado (simular antes de comprar o montar, acceso eficiente a los periféricos) contribuye a reducir el consumo energético o los residuos electrónicos.

- Prototipar y validar el circuito virtualmente en plataformas como Wokwi antes de comprar componentes físicos evita daños irreparables en microcontroladores, LEDs o sensores por cortocircuitos, polarizaciones invertidas o sobrecorrientes. Esto extiende la vida útil del hardware y reduce la generación de basura electrónica derivado de placas estropeadas en fase de prueba.
- La manipulación directa de registros elimina la sobrecarga (*overhead*) de las funciones de alto nivel. Dado que las instrucciones por registro se ejecutan hasta 15 veces más rápido (en apenas 1 a 3 ciclos de reloj), el procesamiento finaliza en un tiempo drásticamente menor, permitiendo que el microcontrolador entre rápidamente en modos de suspensión (*sleep modes*) de bajo consumo de energía

10. Ventajas y limitaciones de simular en Wokwi frente al hardware real (rebote del pulsador, límites de corriente, tiempos).

Ventajas:

Permite prototipar sin costos económicos, validar la lógica de programación y la respuesta temporal de los registros de manera instantánea, y analizar el comportamiento del sistema de forma determinista y segura, sin riesgo de dañar componentes físicos.

Limitaciones:

1. Rebote del pulsador (*Debounce*): En Wokwi el pulsador se simula sin ruido mecánico ("bounce": "0"), mientras que en hardware real se producen conmutaciones parásitas (rebotes) que requieren soluciones por software (retardos/temporizadores) o hardware (filtros RC).
2. Límites de corriente y física: El simulador no considera caídas de voltaje, tolerancias de las resistencias, ni daños por sobrepasar el límite de corriente máximo por pin (40 mA) o por chip.
3. Tiempos y variabilidad real: Wokwi ejecuta instrucciones basándose en un reloj determinista ideal; en hardware real existen derivas térmicas, imprecisiones del cristal oscilador y ruido electromagnético que pueden alterar la temporización.

Enlaces:**Codigo1_BlinkAPI:**

<https://wokwi.com/projects/477197916801108993>

Codigo2_BlinkRegistros:

<https://wokwi.com/projects/477197984180526081>

Codigo3_LaboratorioBits:

<https://wokwi.com/projects/477198025931694081>

Codigo4_MedicionTiempo:

<https://wokwi.com/projects/477198099518687233>

Codigo5_AnalisisEnsamblador

<https://godbolt.org/z/YWYTGoP8f>

GitHub:

https://github.com/DavidCh02/APE_1.git

Diagrama:

```
1 {
2   "version": 1, "author": "Luis Sinche", "editor": "wokwi", "parts": [
3     { "type": "wokwi-arduino-uno", "id": "uno", "top": 0, "left": 0, "attrs": {} },
4     { "type": "wokwi-led", "id": "led0", "top": -170, "left": 140, "attrs": { "color": "red" } },
5     { "type": "wokwi-led", "id": "led1", "top": -170, "left": 180, "attrs": { "color": "yellow" } },
6     { "type": "wokwi-led", "id": "led2", "top": -170, "left": 220, "attrs": { "color": "limegreen" } },
7     { "type": "wokwi-led", "id": "led3", "top": -170, "left": 260, "attrs": { "color": "blue" } },
8     { "type": "wokwi-resistor", "id": "r0", "top": -90, "left": 125, "rotate": 90, "attrs": { "value": "220" } },
9     { "type": "wokwi-resistor", "id": "r1", "top": -90, "left": 165, "rotate": 90, "attrs": { "value": "220" } },
10    { "type": "wokwi-resistor", "id": "r2", "top": -90, "left": 205, "rotate": 90, "attrs": { "value": "220" } },
11    { "type": "wokwi-resistor", "id": "r3", "top": -90, "left": 245, "rotate": 90, "attrs": { "value": "220" } },
12    { "type": "wokwi-pushbutton", "id": "btn1", "top": -150, "left": 330, "attrs": { "color": "green", "bounce": "0" } }
13  ],
14  "connections": [
15    [ "uno:8", "r0:2", "red", [], [ "r0:1", "led0:A", "red", [] ],
16    [ "uno:9", "r1:2", "orange", [], [ "r1:1", "led1:A", "orange", [] ],
17    [ "uno:10", "r2:2", "green", [], [ "r2:1", "led2:A", "green", [] ],
18    [ "uno:11", "r3:2", "blue", [], [ "r3:1", "led3:A", "blue", [] ],
19    [ "led0:C", "uno:GND.1", "black", [], [ "led1:C", "uno:GND.1", "black", [] ],
20    [ "led2:C", "uno:GND.1", "black", [], [ "led3:C", "uno:GND.1", "black", [] ],
21    [ "btn1:1.1", "uno:2", "violet", [], [ "btn1:2.1", "uno:GND.1", "black", [] ]
22  ],
23  "dependencies": {}
24 }
25
```

Conclusiones:

- Se comprobó que la manipulación directa de los registros de $\$E/S\$$ (DDRx, PORTx, PINx) permite configurar y controlar con precisión los pines de entrada y salida del ATmega328P sin depender de la API estándar de Arduino. Asimismo, la aplicación de máscaras de bits mediante operaciones lógicas (AND, OR, XOR) garantizó la modificación selectiva de estados en el puerto (como el encendido/apagado de los LEDs en PB0..PB3) sin alterar la configuración ni el funcionamiento de otros periféricos compartidos en el mismo registro, como el LED integrado en PB5.
- El análisis comparativo de tiempos demostró que el acceso directo a registros reduce drásticamente la latencia y el tiempo de ejecución en comparación con las funciones de alto nivel (digitalWrite). Mientras que la API de Arduino requiere en promedio 3.395 μs por cada conmutación de pin, la escritura por registro directo se ejecuta en tan solo 0.220 μs , ofreciendo un rendimiento aproximadamente 15 veces superior que optimiza el ciclo de trabajo del microcontrolador y minimiza el consumo energético.
- Mediante la inspección del código descompilado, se verificó la traducción directa de las instrucciones en C a su conjunto equivalente en ensamblador AVR. Operaciones atómicas de un solo bit (como sbi o cbi) se completan en apenas 2 ciclos de reloj (125 ns a 16 MHz), mientras que operaciones no atómicas sobre puertos (como el XOR explícito) requieren secuencias de lectura-modificación-escritura (in, eor, out). Esto permitió determinar de forma precisa que una iteración completa del bucle while(1) toma entre 10 y 11 ciclos de reloj (625 ns a 687.5 ns), validando la predecibilidad del hardware ante entornos de tiempo real.

Declaración de uso de Inteligencia Artificial Generativa (IAG)

El uso de herramientas de IAG se ha limitado a funciones de apoyo, sin sustituir el juicio crítico ni la autoría intelectual del autor. En particular, se han utilizado para:

- Mejora de redacción y estilo académico.
- Generación de ideas preliminares.
- Apoyo en la estructuración del documento.
- Asistencia en traducción técnica.

9. Entregables y estructura del informe

- **Carátula:** datos de la práctica e integrantes.
- **Desarrollo por partes (A–E):** capturas legibles del circuito, del Monitor Serie y de la vista I/O de Microchip Studio (o de Compiler Explorer).
- **Tablas 4 y 5 completas,** y la tabla de mediciones de la Parte D con su promedio y la relación calculada.
- **Códigos comentados,** con los enlaces públicos de Wokwi y del repositorio de GitHub.
- **Respuestas a las preguntas de control** y al menos tres **conclusiones técnicas.**
- **Declaración de uso de IA generativa:** herramienta, para qué se usó y qué se verificó; o declaración expresa de no uso.
- **Referencias** en formato APA.

Entrega en el EVA, en PDF, en la fecha indicada en la plataforma. La retroalimentación se publicará en un máximo de 5 días hábiles.

10. Evaluación

Componente: Aprendizaje Práctico-Experimental (APE), Unidad 1. Calificación sobre 10 puntos con la siguiente rúbrica:

Criterio (puntos)	Excelente (100 %)	Satisfactorio (70 %)	Insuficiente (≤ 40 %)
Simulación y configuración de pines (1,5)	Circuito funcional según la Tabla 3; Tabla 4 completa y correcta.	Circuito funcional; errores menores en la Tabla 4.	Circuito incompleto o confusión entre DDR, PORT y PIN.
Programación por registros (2,5)	Códigos 2 y 3 funcionan sin API; pasos B2-B4 y C3-C6 resueltos y explicados.	Funcionan, pero con explicación parcial o uso residual de la API.	No funcionan o dependen de la API de Arduino.
Medición API frente a registros (1,5)	Tres mediciones, promedio, relación y explicación de la causa.	Mediciones correctas con explicación insuficiente.	Sin mediciones o mediciones inconsistentes.
Set de instrucciones (2,0)	Tabla 5 completa con instrucciones y ciclos correctos; uso del simulador evidenciado.	Tabla 5 con uno o dos errores u omisiones.	No identifica la relación entre C y ensamblador.
Preguntas de control y conclusiones (1,5)	Respuestas correctas, argumentadas con evidencia de la práctica.	Respuestas mayormente correctas, poco argumentadas.	Respuestas incompletas o incorrectas.
Informe y entrega (1,0)	Estructura completa, capturas legibles, repositorio, declaración de IA y entrega puntual.	Falta un elemento de la estructura.	Faltan varios elementos o entrega tardía.

11. Bibliografía

- Freire Sánchez, L. A. (2023). *Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico*. ISBN 978-9942-636-22-5.
- Valdivieso, C., y Solís, R. (2019). *Microprocesadores: fundamentos y aplicaciones*. LATIn Project.
- Zailani, N. F. (2024). *Getting started with Arduino and ESP32: A beginner's guide with Wokwi*. Politeknik Mersing.
- Microchip Technology Inc. (2024). *ATmega328P – hoja de datos y documentación del producto*. <https://www.microchip.com/en-us/product/ATmega328P>
- Microchip Technology Inc. (s. f.). *AVR® Instruction Set Manual*. <https://www.microchip.com>
- CodeMagic LTD. (2025). *Wokwi – documentación*. <https://docs.wokwi.com>
- Arduino. (2024). *Arduino Language Reference*. <https://www.arduino.cc/reference/>



12. Elaboración y Aprobación

Elaborado por	Ing. Luis Darío Sinche Cueva Docente	 Validar únicamente en FirmaEC. Firmado electrónicamente por: LUIS DARIO SINCHE CUEVA
Revisado por	Ing. Ramiro Rene Rivera Guamán Técnico Docente	 Validar únicamente en FirmaEC. Firmado electrónicamente por: RAMIRO RENE RIVERA GUAMAN
Aprobado por	Ing. Edison Leonardo Coronel Romero Director de Carrera	 Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC

Anexo A. diagram.json del circuito de la Parte C (Wokwi)

Copie este contenido en la pestaña `diagram.json` del proyecto de Wokwi. El archivo también está disponible en el EVA.

```
{
  "version": 1, "author": "Luis Sinche", "editor": "wokwi",
  "parts": [
    {"type": "wokwi-arduino-uno", "id": "uno", "top": 0, "left": 0, "attrs": {}},
    {"type": "wokwi-led", "id": "led0", "top": -170, "left": 140, "attrs": {"color": "red"}},
    {"type": "wokwi-led", "id": "led1", "top": -170, "left": 180, "attrs": {"color": "yellow"}},
    {"type": "wokwi-led", "id": "led2", "top": -170, "left": 220, "attrs": {"color": "limegreen"}},
    {"type": "wokwi-led", "id": "led3", "top": -170, "left": 260, "attrs": {"color": "blue"}},
    {"type": "wokwi-resistor", "id": "r0", "top": -90, "left": 125, "rotate": 90, "attrs": {"value": "220"}},
    {"type": "wokwi-resistor", "id": "r1", "top": -90, "left": 165, "rotate": 90, "attrs": {"value": "220"}},
    {"type": "wokwi-resistor", "id": "r2", "top": -90, "left": 205, "rotate": 90, "attrs": {"value": "220"}},
    {"type": "wokwi-resistor", "id": "r3", "top": -90, "left": 245, "rotate": 90, "attrs": {"value": "220"}},
    {"type": "wokwi-pushbutton", "id": "btn1", "top": -150, "left": 330, "attrs": {"color": "green", "bounce": "0"}}
  ],
  "connections": [
    ["uno:8", "r0:2", "red", []], ["r0:1", "led0:A", "red", []],
    ["uno:9", "r1:2", "orange", []], ["r1:1", "led1:A", "orange", []],
    ["uno:10", "r2:2", "green", []], ["r2:1", "led2:A", "green", []],
    ["uno:11", "r3:2", "blue", []], ["r3:1", "led3:A", "blue", []],
    ["led0:C", "uno:GND.1", "black", []], ["led1:C", "uno:GND.1", "black", []],
    ["led2:C", "uno:GND.1", "black", []], ["led3:C", "uno:GND.1", "black", []],
    ["btn1:1.1", "uno:2", "violet", []], ["btn1:2.1", "uno:GND.1", "black", []]
  ],
  "dependencies": {}
}
```